

Operating Systems

Homework 1: File operations

Due date (via moodle): **7th of May, 2026, 23:59**

Individual work policy

The work you submit in this course is required to be the result of your individual effort **only**. You may discuss concepts and ideas with others, but **you must work individually**. **You should never observe another student's solutions**, from this or previous semesters.

Students violating this policy will receive appropriate sanctions.

AI Usage Policy

Section 2 (Programming Task): The use of AI tools (ChatGPT, Copilot, Claude, etc.) is **strictly prohibited** for writing the code. This part must be completed entirely by you.

Section 3 (Performance Analysis): You are **required** to use an AI assistant as specified in the assignment to investigate the `O_EXCL` flag and compare it with official documentation.

Section 4 (AI Experiment): You are **required** to use an AI chatbot to attempt solving the programming problem and document the process.

The purpose of this exercise is to get refreshed with C basic programming. Then, we will practice writing and running a simple program in Unix/Linux to practice using the basic file access system calls.

If you didn't install the virtual machine (VM) from the Moodle of the course, please do it before you start this exercise. **You should run this exercise on the VM.**

1 Background and Review

1.1 Basic C Program Structure

First, recall the basic C program structure, including handling command-line arguments:

```
1 #include <stdio.h>
2 ...
3 int main(int argc, char* argv[])
4 {
5     if (argc < 2) { /* assuming the program needs 1 argument */
6         fprintf(stderr, "usage: %s <arg>\n", argv[0]);
7         exit(1); /* stop the program; report error */
8     }
9     ...
10    return 0; /* successful return */
11 }
```

1.2 File Operations

In this section, we will review the system calls we learned in recitation 1, along with a few others. System calls are documented in Linux in so-called man (manual) pages. The manual has several sections; system calls are documented in Section 2. To read the documentation for any system call, use the `man` command in your shell, specifying section 2. For example, to learn about the `open` system call, run:

```
man 2 open
```

We strongly encourage you to read the `man` pages for all system calls used in this assignment, paying close attention to their parameters and return values.

1.2.1 Opening a file:

```
1 #include <sys/types.h>
2 #include <sys/stat.h>
3 #include <fcntl.h>
4 ...
5 int open(char* filename, int flags, int mode);
```

The first argument is the path name of the file to open, for example: `/data/my_file.txt`.

The second argument describes how you want to open the file, using the following options, and a few more that we do not show (you can specify multiple options using the `or` operation - `|`):

1. `O_RDONLY` - read only
2. `O_WRONLY` - write only
3. `O_RDWR` - read and write
4. `O_APPEND` - write will always be performed at the end of the file
5. `O_CREAT` - create a file if it doesn't exist
6. `O_TRUNC` - the content of an existing file will be deleted and its size will be truncated to 0
7. `O_SYNC` - every write will be immediately written to disk

There are other less commonly used options.

The third argument is only needed when creating a new file and specifies the permissions for the file (you can also specify the permission number using the `or` operation):

1. `S_IRUSR` - the user can read
2. `S_IWUSR` - the user can write
3. `S_IXUSR` - the user can execute the file
4. `S_IRGRP` - group read permission
5. `S_IROTH` - others read permission, etc.

The third argument (`mode`) will be discussed later on the course. For now you can set it to: `S_IRUSR | S_IWUSR | S_IXUSR`.

The returned identifier (**file descriptor**) is used for file access. If the value -1 is returned, the file opening failed, and the value of the global variable `errno` indicates the reason for failure. The error codes are explained in the man page of `open`. Also see `man errno`.

1.2.2 Closing a file:

```
1 #include <unistd.h>
2 ...
3 int close(int fd);
```

The parameter is the identifier returned by `open`. The function returns an error code: 0 represents success and -1 failure. The only possible failure is `errno==EBADF` (an identifier that doesn't represent an open file). The file is closed with all its changes written to disk.

1.2.3 Reading and Writing:

```
1 #include <unistd.h>
2 ...
3 ssize_t read(int fd, void* buf, size_t count);
4 ssize_t write(int fd, void* buf, size_t count);
```

The first parameter specifies a file using the identifier returned from `open`. The second parameter specifies a memory address from/to which the data will be transferred. The third parameter specifies how many bytes to transfer. The functions return the number of bytes actually transferred or -1 if an error occurred (the `ssize_t` type is a signed integer type defined in `unistd.h`).

For each open file, there is a pointer that defines the location (in bytes) in the file where the next read or write will be performed. Each read and write operation advances the pointer by the number of bytes transferred. When a file is opened, the pointer points to the beginning of the file.

1.2.4 Moving the pointer (or clarifying where it is):

```
1 #include <sys/types.h>
2 #include <unistd.h>
3 ...
4 off_t lseek(int fd, off_t offset, int whence);
```

The first parameter specifies an open file. The second parameter specifies where to move the pointer (number of bytes). The third parameter specifies whether the movement (second parameter) is from the beginning of the file, from the end of the file, or from the current position. The value of the third parameter can be:

1. `SEEK_SET` - from the beginning of the file
2. `SEEK_END` - from the end
3. `SEEK_CUR` - from the current position

The returned value is the current position (after the movement) in bytes from the beginning of the file, or -1 if an error occurred.

1.2.5 Notes about `fopen`, `fread`, `fwrite`, `fseek`:

The functions `fopen`, `fread`, `fwrite`, `fseek` provide similar services to the system calls described here but they are part of the C standard library and not direct system calls. Using them makes a program more portable but you cannot perform every operation that can be performed through the direct interface to the operating system (due to differences between operating systems). In this course, we use the direct interface to the operating system and not auxiliary libraries.

2 Programming Task

Write/Code a program that copies a file's content to another file. Name your source file `copy.c`.

The program should print an error message and exit if there aren't enough parameters to the program, if the **input** file doesn't exist, or if the **output** file exists. In addition, your program must handle errors from system calls as described in the **Error Handling** subsection below.

The program should use the system calls described in this exercise, besides `lseek`. The copying should be done using a buffer of size n bytes, meaning the program reads n bytes in one read, writes them, and continues in a loop.

Error Handling

Your program must check for errors and handle them appropriately. There are two types of errors to handle:

1. **Logic/input errors** (e.g., wrong number of arguments). Use `fprintf(stderr, ...)` to print an error message. For example:

```
if (argc < 4) {
    fprintf(stderr, "not enough arguments");
    exit(1);
}
```

2. **System call failures** (e.g., `open`, `read`, `write` returning -1): use `perror()` to print a descriptive error message that includes the `errno` description. For example:

```
int fd = open("file.txt", O_RDONLY);
if (fd == -1) {
    perror("open");
    exit(1);
}
```

In both cases, your program should also clean up any resources (close open files, free allocated memory) before calling `exit(1)`.

The program has three parameters: input file name, output file name, and n . To convert the string representation of n to an integer, you can use `sscanf(argv[3], "%d", &n)`. Use `malloc` to allocate memory for the buffer:

```
1 #include <stdlib.h>
2 ...
3 char* buffer;
4 ...
5 buffer = (char*) malloc(n);
```

3 Performance Analysis

Now that your program is working, let's analyze how buffer size affects performance:

Important: Avoiding the File System Cache

A modern OS will cache files in memory (the "page cache") after they are read once. This makes subsequent reads extremely fast (reading from RAM instead of disk) and will **invalidate your measurements**.

To get accurate results for this I/O experiment, you must force the OS to clear this cache before *every single run*. You can clear the cache on Linux with this command (it requires **sudo**):

```
sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
```

Experiment Steps

1. **Create a large test file.** Use a file of at least **100MB** to ensure the disk I/O time is significant. You can use the **dd** command:

```
1 # Creates a 100MB file of random data
2 dd if=/dev/urandom of=testfile bs=1M count=100
```

2. **Measure execution time.** You will test your program with the following buffer sizes (in bytes): 1, 64, 512, 1024, 8192, and 65536. For *each* buffer size, you must first clear the cache and then immediately run your program using **time**.

Example for the 1024-byte test:

```
1 # 1. Clear the cache
2 sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
3
4 # 2. Immediately run the test
5 time ./copy testfile outputfile 1024
```

The **time** command will show three values: **real**, **user**, and **sys**. For this analysis, focus on the **real** (wall-clock) time.

3. Record the runtime for each buffer size and create a table or graph showing your results.
4. Analyze your measurements: Is there a noticeable difference in runtime between the different buffer sizes? If so, explain why smaller or larger buffers might perform better or worse. Write a brief report that includes your timing measurements, analysis, and conclusions. Submit it as a PDF file.
5. **Within your PDF report**, you must also include a section addressing the following points:
 - Investigate and explain what the **O_EXCL** option for **open** does. Your explanation must be based on a summary and comparison of the information you find from two sources:
 - (a) The official documentation (using **man 2 open**).
 - (b) An AI chat assistant (e.g., ChatGPT, Claude, Gemini, etc).
 - An explanation of what happens when you give the shell command **man open** (without the '2') and why this output differs from **man 2 open**.

4 AI Experiment: Observing AI Capabilities and Limitations

Now that you have completed the programming task yourself, conduct an experiment to observe how AI tools approach the same problem:

1. Choose an AI chat assistant (e.g., ChatGPT, Claude, Gemini, etc).
2. Craft a prompt that asks the AI to solve the programming task from Section 2. Start with the shortest, simplest prompt possible.
3. Evaluate the generated program:
 - Does it compile without errors or warnings?
 - Does it meet all the requirements specified in Section 2?
 - Does it handle errors correctly?
 - Does it work correctly when tested?
4. If the program is flawed or incomplete, continue the conversation, asking the AI chat assistant that you chose to fix the specific problems. Keep refining until you obtain a program that fully satisfies all requirements and runs correctly.
5. **In your PDF report**, include a section titled "AI Experiment" that contains:
 - The name of the AI chat assistant you used
 - Your initial prompt
 - Any follow-up prompts you needed to give
 - A brief reflection: What did the AI get right immediately? What did it miss? How many iterations were needed?

Important: Do **NOT** submit the AI-generated program. Only include the prompts and reflection in your PDF. The code you submit in `copy.c` must be your own work from Section 2.

5 Compiling Requirements

Your program must compile cleanly on the course VM (no errors or warnings) when the following command is run:

```
gcc -O3 -Wall -std=c11 copy.c -o copy
```

Submission Requirements

Submit a **single ZIP file** named `hw1_XXXXXXXXX.zip` (where XXXXXXXXXX is your ID number) containing all deliverables from *all* parts of HW1. Meaning, your ZIP file must contain the following files:

1. `copy.c` (Your C source file)
2. `hw1_report.pdf` (Your PDF analysis report, including performance analysis, `O_EXCL` investigation, and the AI experiment)